

Fast and simple algorithms to count the number of vertex covers in an interval graph

Min-Sheng Lin

Department of Electrical Engineering, National Taipei University of Technology, 1, Sec. 3, Chung-Hsiao E. Rd., Taipei 106, Taiwan, ROC

Received 20 July 2006; received in revised form 30 November 2006

Available online 20 December 2006

Communicated by M. Yamashita

Abstract

This study provides the following fast and simple algorithms for an interval graph: (1) an algorithm for counting the number of vertex covers, and (2) an algorithm for counting the number of minimal vertex covers.

© 2006 Elsevier B.V. All rights reserved.

Keywords: Algorithms; Interval graph; Vertex cover; Independent set; Counting

1. Introduction

Let $G = (V, E)$ denote a simple graph with vertex set V and edge set E . A *vertex cover* in G is a subset C of V such that every edge in E has at least one endpoint in C . A vertex cover C is called *minimal* if and only if no proper subset of C is a vertex cover. Independent sets and vertex covers are well known to be closely related objects. An *independent set* in G is a subset D of V such that every edge in E is incident to at most one vertex in D . An independent set D is said to be *maximal* if and only if no proper superset of D is an independent set. Clearly, a set C is a (minimal) vertex cover of G if and only if $V - C$ is a (maximal) independent set of G . Thus, the number of (minimal) vertex covers in G simply equals the number of (maximal) independent sets in G . However, counting the number of vertex cover (independent set) is a #P-complete problem, even when

the graph is restricted to bipartite graphs [1]. Valiant [2] defined the class of #P-complete problems. The class of #P problems includes problems that involve counting access computations for problems in NP, while the class of #P-complete problems includes the hardest problems in #P. As is widely recognized, all known exact algorithms for solving these problems have exponential time complexity, making efficient algorithms for this class of problems unlikely to be developed. However, this complexity can be mitigated by considering only a restricted subclass of #P-complete problems. Okamoto et al. [3] gave $O(|V| + |E|)$ time algorithms for counting the number of independent sets in a chordal graph, and also showed that the problem of counting the number of maximal independent sets remains #P-complete in a chordal graph.

This study considers a subclass of chordal graphs—interval graphs. An undirected graph G is called an interval graph if its vertices can be placed into a one-to-one correspondence with a family I of intervals on the real line such that two vertices of G are connected

E-mail address: mslin@ee.ntut.edu.tw (M.-S. Lin).

by an edge if and only if the corresponding intervals of I have a intersection. Many algorithms have been developed for interval graphs, which have applications in various fields. This work addresses the problems of counting the number of vertex covers and the number of minimal vertex covers in an interval graph.

2. An $O(|V|)$ -time algorithm to count the number of vertex covers

This section presents an efficient algorithm for counting the number of vertex covers in an interval graph. Let I denote an interval representation of graph $G = (V, E)$ where $|V| = n$ and $V = \{1, 2, \dots, n\}$. For simplicity, the interval in I corresponding to vertex k in V is called interval k . Let $a(k)$ and $b(k)$ represent the left and right endpoints of interval k , respectively. Without loss of generality, assume that no two intervals share a common endpoint. The endpoints of all intervals are labeled with distinct integers between 1 and $2n$. The vertices from 1 to n are labeled according to their ascending right endpoints. That is, for two vertices i and j , $b(i) < b(j)$ if and only if $i < j$. Let V_k be the set $\{1, 2, \dots, k\}$ of vertices with $k \leq n$. The subgraph induced by V_k is denoted by G_k . $N_k(i)$ represents the (open) neighborhood of vertex i in G_k , and $N_k[i] = \{i\} \cup N_k(i)$ represents the closed neighborhood of vertex i in G_k . Let $x(k)$ be the largest vertex on the left side of vertex k and not adjacent to vertex k , or equivalently, $x(k) = \max\{i \mid b(i) < a(k)\} = k - |N_k[k]|$. This implies that $N_k(k) = \{k-1, k-2, \dots, x(k)+2, x(k)+1\}$. The following algorithm can be applied to compute $x(k)$, for $1 \leq k \leq n$, in $O(n)$ time.

Algorithm Compute_x

```

begin
   $x(0) \leftarrow 0$ ; // initial condition //
   $h \leftarrow 0$ ; //  $h$  is the rightmost interval for now //
  for  $s \leftarrow 1$  to  $2n$  do
    if  $s = a(k)$  then  $x(k) \leftarrow h$ ;
      //  $s$  is the left end-point of interval  $k$  //
    if  $s = b(k)$  then  $h \leftarrow k$ ;
      //  $s$  is the right end-point of interval  $k$  //
  end-for
end-algorithm

```

Let $vc(k)$ denote the number of vertex covers in G_k . Theorem 2.1 presents a recurrence equation to determine the number of vertex covers in an interval graph.

Theorem 2.1. For $1 \leq k \leq n$, $vc(k) = vc(k-1) + vc(x(k))$.

Proof. Let C denote a vertex cover of G_k .

Case 1: Vertex $k \in C$. Clearly, C is a vertex cover with vertex $k \in C$ if and only if $C - \{k\}$ is a vertex cover of G_{k-1} . Thus, the number of vertex covers with vertex $k \in C$ is given by $vc(k-1)$.

Case 2: Vertex $k \notin C$. According to the vertex cover, $N_k(k)$ is a subset of C and $C - N_k(k)$ is a vertex cover of $G_{x(k)}$. Conversely, for any vertex cover C' of $G_{x(k)}$, $C' \cup N_k(k)$ is a vertex cover of G_k . Therefore, the number of vertex covers with vertex $k \notin C$ is given by $vc(x(k))$. Hence, Theorem 2.1 holds. $\square$

The $O(n)$ time algorithm is described in detail as follows.

Algorithm Count_vc

```

begin
   $vc(0) \leftarrow 1$ ; // initial condition //
  for  $k \leftarrow 1$  to  $n$  do
     $vc(k) \leftarrow vc(k-1) + vc(x(k))$ ;
    // by Theorem 2.1 //
  end-for
  output("the number of vertex cover is",  $vc(n)$ );
end-algorithm

```

3. An $O(|V|)$ -time algorithm to count the number of minimal vertex covers

This section develops an efficient algorithm for counting the number of minimal vertex covers in an interval graph. Let M_k denote the family of all minimal vertex cover sets in G_k . For vertices i, j , and k with $i < j \leq k$, let $M_k[j, \bar{i}]$ represent the set of all minimal vertex cover sets in G_k including j but not including i . Since $N_k[k] = \{k, k-1, k-2, \dots, x(k)+1\}$, we have

$$\begin{aligned}
 M_k &= M_k[\bar{k}] \cup M_k[k, \overline{k-1}] \\
 &\quad \cup M_k[k, k-1, \overline{k-2}] \cup \dots \\
 &\quad \cup M_k[k, k-1, \dots, \overline{x(k)+1}] \\
 &\quad \cup M_k[k, k-1, \dots, x(k)+1].
 \end{aligned}$$

Significantly, the last term $M_k[k, k-1, \dots, x(k)+1]$ is an empty set since no minimal vertex cover includes k and all neighbors of k $\{k-1, k-2, \dots, x(k)+1\}$. Moreover, the terms in the right side of the above equation are disjoint to each other. Thus, we have

$$\begin{aligned}
 |M_k| &= |M_k[\bar{k}]| + |M_k[k, \overline{k-1}]| \\
 &\quad + |M_k[k, k-1, \overline{k-2}]| \\
 &\quad + \dots + |M_k[k, k-1, \dots, \overline{x(k)+1}]|. \quad (3.1)
 \end{aligned}$$

The general term in Eq. (3.1) is derived using the following lemma.

Lemma 3.1. For $k - 1 \geq h \geq x(k) + 1$,

$$|M_k[k, k - 1, \dots, h + 1, \bar{h}]| = \begin{cases} |M_h[\bar{h}]| & \text{if vertex } h \text{ is adjacent to all vertices} \\ & \text{in } \{k, k - 1, \dots, h + 1\}, \\ 0 & \text{otherwise.} \end{cases}$$

Proof. *Case 1:* Vertex h is adjacent to all vertices in $\{k, k - 1, \dots, h + 1\}$.

(“ $\supseteq$ ” part) Let S represent a minimal vertex cover of G_h not including h , i.e., $S \in M_h[\bar{h}]$. Since h is adjacent to all vertices in $\{k, k - 1, \dots, h + 1\}$, $S' = S \cup \{k, k - 1, \dots, h + 1\}$ is a minimal vertex cover of G_k . Thus, $S' \in M_k[k, k - 1, \dots, h + 1, \bar{h}]$.

(“ $\subseteq$ ” part) Let S be a minimal vertex cover of G_k including $\{k, k - 1, \dots, h + 1\}$ but not including $\{h\}$, i.e., $S \in M_k[k, k - 1, \dots, h + 1, \bar{h}]$. Clearly, $S' = S - \{k, k - 1, \dots, h + 1\}$ is a vertex cover of G_h not including h . Assume that S' is not a minimal vertex cover, i.e., $S' \notin M_h[\bar{h}]$. A vertex $v \in S'$ must exist such that $S' - \{v\}$ is also a vertex cover of G_h . Therefore, $S - \{v\}$ is a vertex cover of G_k including $\{k, k - 1, \dots, h + 1\}$ but not including $\{h\}$. This contradicts the statement that S is a minimal vertex cover in $M_k[k, k - 1, \dots, h + 1, \bar{h}]$. Hence, S' is a minimal vertex cover of G_h not including h , i.e., $S' \in M_h[\bar{h}]$.

Case 2: A vertex i exists such that i is not adjacent to h , for $k - 1 \geq i > h \geq x(k) + 1$.

Since vertex i is not adjacent to h and $i > h$, by the definition of interval graph and the order of intervals, i is not adjacent to any vertex in $\{h - 1, h - 2, \dots, 1\}$. Thus, $N_k(i) \subseteq \{k, k - 1, \dots, h + 1\}$. Assume that $S \in M_k[k, k - 1, \dots, h + 1, \bar{h}]$ exists. Since $N_k(i) \subseteq \{k, k - 1, \dots, h + 1\}$, $S' = S - \{i\}$ is also a vertex cover of G_k . This is a contradiction to S being a minimal vertex cover. Therefore, $M_k[k, k - 1, \dots, h + 1, \bar{h}] = \emptyset$ and $|M_k[k, k - 1, \dots, h + 1, \bar{h}]| = 0$ $\square$

Corollary 3.1. For G_k , if vertices i and h exist such that i is not adjacent to h and $k - 1 \geq i > h \geq x(k) + 1$, then

$$\begin{aligned} & |M_k[k, k - 1, \dots, i, \dots, \bar{h}]| \\ &= |M_k[k, k - 1, \dots, i, \dots, \overline{h - 1}]| = \dots \\ &= |M_k[k, k - 1, \dots, i, \dots, \overline{x(k) + 2, \overline{x(k) + 1}}]| = 0. \end{aligned}$$

Proof. Since i is not adjacent to h for $i > h$, then i is not adjacent to any vertex in $\{h - 1, h - 2, \dots, x(k) + 1\}$. According to Lemma 3.1, the corollary holds. $\square$

Let $y(k)$ denote the maximal value satisfying the condition of h in Corollary 3.1, i.e., $y(k) = \max\{x(i) \mid i \in N_k(k)\}$ and $y(k) \geq x(k) + 1$. In particular, if no

such $y(k)$ exists, i.e., $\max\{x(i) \mid i \in N_k(k)\} < x(k) + 1$, then $y(k) = x(k)$. Therefore, $y(k)$ can be redefined as $\max\{x(i) \mid i \in N_k[k]\}$. The following lemma and algorithm yield a recurrence equation to determine $y(k)$.

Lemma 3.2. For $1 \leq k \leq n$, $y(k) = \max(x(k), y(k - 1))$.

Proof. *Case 1:* $y(k - 1) > x(k)$.

Let $x(h) = y(k - 1) > x(k)$. The definition of $y(k - 1)$ gives $h \in N_{k-1}[k - 1]$, and hence $h \leq k - 1$. Consequently, $x(k) < x(h) < h \leq k - 1 < k$, which implies that $h \in N_k(k)$. Next, show that $x(h) = \max\{x(i) \mid i \in N_k(k)\}$. Assume that $h' \in N_k(k)$ such that $x(h') > x(h)$. Since $h' \in N_k(k)$, we have $h' \leq k - 1$. Since $x(h) = y(k - 1)$, we have $x(h) \geq x(k - 1)$. Hence, $x(k - 1) \leq x(h) < x(h') < h' \leq k - 1$, which implies that $h' \in N_{k-1}[k - 1]$. Since $x(h') > x(h)$, $x(h)$ is not the maximum of $\{x(i) \mid i \in N_{k-1}[k - 1]\}$. This contradicts $x(h) = y(k - 1)$. Therefore, no $h' \in N_k(k)$ exists such that $x(h') > x(h)$, and hence $x(h) = \max\{x(i) \mid i \in N_k(k)\}$. Furthermore, $x(h) = y(k - 1) > x(k)$, so, $x(h) = \max\{x(i) \mid i \in N_k[k]\}$. Hence, $x(h) = y(k) = y(k - 1)$.

Case 2: $y(k - 1) \leq x(k)$.

Assume that $h' \in N_k(k)$ exists such that $x(h') > x(k)$. Since $x(k - 1) \leq y(k - 1) \leq x(k)$ and $x(k) < x(h') < h' \leq k - 1$, we have $x(k - 1) < h' \leq k - 1$, which implies that $h' \in N_{k-1}[k - 1]$ and $x(h') \leq \max\{x(i) \mid i \in N_{k-1}[k - 1]\}$. Since $x(h') > x(k)$, we have $x(k) < \max\{x(i) \mid i \in N_{k-1}[k - 1]\} = y(k - 1)$, and a contradiction occurs. Therefore, no $h' \in N_k(k)$ exists such that $x(h') > x(k)$. Thus, $y(k) = x(k)$. $\square$

Algorithm Compute_ y

begin

$y(0) \leftarrow -1$; // initial condition //

for $k \leftarrow 1$ **to** n **do**

if $y(k - 1) > x(k)$ **then** $y(k) \leftarrow y(k - 1)$

else $y(k) \leftarrow x(k)$; // by Lemma 3.3 //

end-for

end_algorithm

By Corollary 3.1,

$$\begin{aligned} & M_k[k, k - 1, \dots, \overline{y(k)}] \\ &= M_k[k, k - 1, \dots, y(k), \overline{y(k) - 1}] = \dots \\ &= M_k[k, k - 1, \dots, x(k) + 2, \overline{x(k) + 1}] = \emptyset. \end{aligned}$$

According to Lemma 3.1, Eq. (3.1) can be rewritten as

$$\begin{aligned}
|M_k| &= |M_k[\bar{k}]| + |M_k[k, \overline{k-1}]| + \dots \\
&\quad + |M_k[k, k-1, \dots, \overline{y(k)+1}]| \\
&= |M_k[\bar{k}]| + |M_{k-1}[\overline{k-1}]| + \dots \\
&\quad + |M_{y(k)+1}[\overline{y(k)+1}]|.
\end{aligned}$$

From the proof of case 2 in Theorem 2.1, clearly, $|M_k[\bar{k}]| = |M_{x(k)}|$ for $1 \leq k \leq n$. Let $mvc(k)$ denote the cardinality of M_k . The following lemma holds.

Lemma 3.3. For $1 \leq k \leq n$,

$$mvc(k) = \sum_{i=y(k)+1}^k mvc(x(i)).$$

By Lemma 3.3, since

$$mvc(k-1) = \sum_{i=y(k-1)+1}^{k-1} mvc(x(i)),$$

the following theorem and algorithm are valid.

Theorem 3.1. For $1 \leq k \leq n$, $mvc(k) = mvc(k-1) + mvc(x(k)) - \sum_{i=y(k-1)+1}^{y(k)} mvc(x(i))$.

Algorithm Count_mvc

begin

$mvc(0) \leftarrow 1$; // initial condition //

for $k \leftarrow 1$ **to** n **do**

$mvc(k) \leftarrow mvc(k-1) + mvc(x(k))$;

// by Theorem 3.1 //

for $i \leftarrow y(k-1) + 1$ **to** $y(k)$ **do**

$mvc(k) \leftarrow mvc(k) - mvc(x(i))$

// by Theorem 3.1 //

end-for

end-for

output(“the number of minimal vertex covers is”,
 $mvc(n)$);

end-algorithm

The computational complexity of the above algorithm is

$$O\left(\sum_{k=1}^n \left(\sum_{i=y(k-1)+1}^{y(k)} 1\right)\right) = O(y(n)) = O(n).$$

References

- [1] J.S. Provan, M.O. Ball, The complexity of counting cuts and of computing the probability that a graph is connected, SIAM J. Computing 12 (4) (1983) 777–788.
- [2] L.G. Valiant, The complexity of enumeration and reliability problems, SIAM J. Computing 8 (1979) 410–421.
- [3] Y. Okamoto, T. Uno, R. Uehara, Linear-time counting algorithms for independent sets in chordal graphs, in: Proceedings of 31st International Workshop on Graph-Theoretic Concepts in Computer Science, 2005, pp. 433–444.